AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

SPRING 2025-2026

PROGRAMMING IN PYTHON [B]

Mid-Term Project

Project Report On:

Banking Management System

Submitted To:
MD. TANZEEM RAHAT

Submitted by:
Group: 8

Student ID	Name
22-48152-2	DIAMOND HALDER
22-47992-2	BADHON KUMAR BISWAS
22-48052-2	ANTARA SAHA

1. Project Overview

The Banking Management System (OOP Pro) is a terminal-based application developed using Python and Object-Oriented Programming (OOP) concepts. The system simulates real-world banking operations such as account creation, secure login, transactions, and reporting.

The application provides a menu-driven interface where users can easily navigate through options including:

- Creating a new bank account
- Logging in securely with PIN verification
- Performing financial transactions (deposit, withdraw, transfer)
- Viewing account analytics and reports
- Managing user profiles

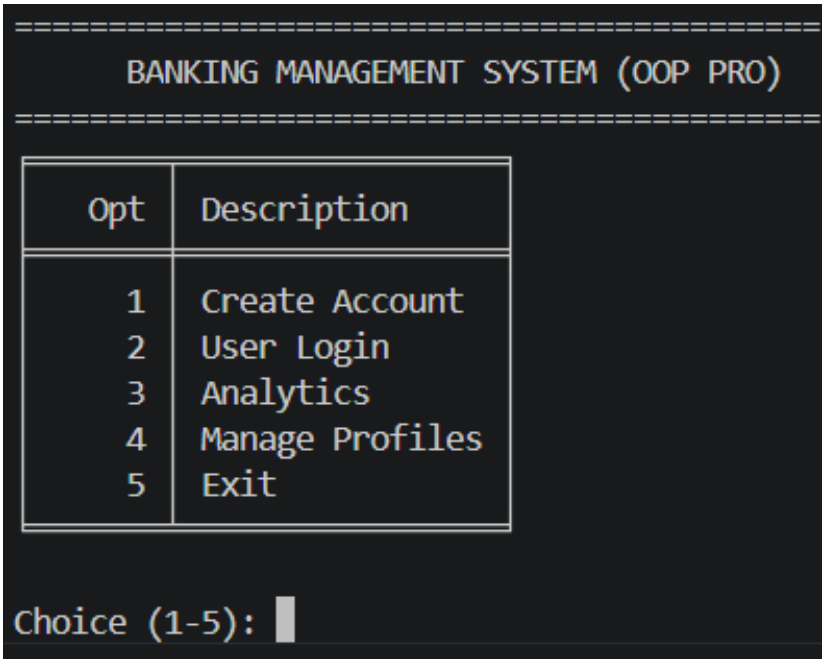

Fig: First Interface — Main Menu

2. Execution Instructions

Follow these steps to set up and run the project from scratch:

1. Open terminal in the project folder and create virtual environment: python -m venv .venv
2. Activate virtual environment: .venv\Scripts\activate
3. Install dependencies: pip install -r requirements.txt
4. Run the project: python main.py

```
PS C:\xampp\htdocs\Banking_System> python -m venv .venv
PS C:\xampp\htdocs\Banking_System> .\.venv\Scripts\Activate
(.venv) PS C:\xampp\htdocs\Banking_System> pip install -r requirements.txt
Requirement already satisfied: tabulate==0.10.0 in c:\xampp\htdocs\banking_system\.venv\lib\site-packages (from -r requirements.txt (line 1)) (0.10.0)

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\xampp\htdocs\Banking_System> python main.py
```

Fig: Terminal — Successful Project Setup and Run

3. Object-Oriented Design (OOP)

The project follows a modular and layered OOP architecture, ensuring clean code separation and maintainability.

Class / Module	Responsibility
`Account`	Entity class — stores user data: account number, name, phone, PIN, balance
`AccountManager`	Service class — handles Create, Update, and Close account operations
`TransactionManager`	Service class — manages Deposit, Withdraw, and Transfer operations
`SecurityManager`	Service class — handles Login and PIN verification logic
`DataStorage`	Storage class — handles CSV file read/write for data persistence
`main.py`	Controller — drives the entire system using a menu-driven loop

4. Key Features

The system includes the following functionalities across its module layers:

Feature	Description
Account Creation & Login	Create accounts with secure PIN-based authentication
Balance Check	View current balance at any time
Deposit & Withdraw	Add or withdraw funds with validation
Fund Transfer	Transfer between accounts with existence check
Transaction History	Full log of all past transactions

PIN Change	Secure PIN update functionality
Profile Management	Update or close account details
Analytics	Bank summary and top customer reports

4.1 Account Creation

```
==============================
   CREATE NEW ACCOUNT
==============================
Enter Customer Name: Badhon
Enter Phone Number: 0171111111
Error: Phone number must be numeric and contain exactly 11 digits.
Enter Phone Number: 01711111111
Set 4-Digit Security PIN: 123
Error: PIN must be exactly 4 digits.
Set 4-Digit Security PIN: 1234
Initial Deposit Amount (Min 500): 50
Error: Value must be at least 500
Initial Deposit Amount (Min 500): 500

Success! Account Number: 1005
```

Fig: Create Account

4.2 User Login

```
-------------------------
   SECURE USER LOGIN
-------------------------
Account Number: 100000
PIN: 123

Error: Invalid credentials!
```

Fig: User Login

```
-------------------------
   SECURE USER LOGIN
-------------------------
Account Number: 1001
PIN: 1234

Welcome back, Badhon Kumar!

>>> Active: Badhon Kumar
1  Balance
2  Deposit
3  Withdraw
4  Transfer
5  History
6  Change PIN
7  Logout

Action:
```

Fig: User Dashboard after Login

4.3 Balance Check

```
==============================
      BALANCE INQUIRY
==============================
Holder: Badhon Kumar
Acc No: 1001
Balance: 4400.00 BDT
==============================
```

Fig: Balance Check

4.4 Deposit

```
Enter Deposit Amount: 500
Deposited 500.0 BDT.

═══════════════════════════════════════
        TRANSACTION SUCCESSFUL
═══════════════════════════════════════
Account Holder     Badhon Kumar
Account No         1001
Transaction Type   Deposit
Amount             500.00 BDT
Current Balance    4900.00 BDT
═══════════════════════════════════════
```

Fig: Deposit

4.5 Withdraw

```
Enter Withdrawal Amount: 5000
Error: Insufficient funds!

>>> Active: Badhon Kumar
1  Balance
2  Deposit
3  Withdraw
4  Transfer
5  History
6  Change PIN
7  Logout

Action:
```

Fig: Withdraw

4.6 Fund Transfer

```
Enter Receiver Acc No: 1002
Transfer to Biswas. Amount: 300
Enter PIN to confirm: 1234
Transfer Successful!

>>> Active: Badhon Kumar
1  Balance
2  Deposit
3  Withdraw
4  Transfer
5  History
6  Change PIN
7  Logout

Action:
```

Fig: Fund Transfer

4.7 Transaction History

--- TRANSACTION HISTORY (ACC: 1001) ---

Date & Time	Type	Amount
28-03-2026 21:39:43	Withdraw	500.00 BDT
28-03-2026 21:40:45	To 1002	1000.00 BDT
28-03-2026 23:09:25	Deposit	500.00 BDT
28-03-2026 23:11:13	To 1002	300.00 BDT

Fig: Transaction History

5. Validation and Robustness

To ensure system stability and a user-friendly experience, the following validations are enforced:

- ✓ Empty input is not allowed
- ✓ PIN must be exactly 4 digits
- ✓ Phone number must be fully numeric
- ✓ Withdrawal is blocked if balance is insufficient
- ✓ Transfer fails if the receiver account does not exist
- ✓ Account closure only permitted when balance is zero and correct PIN is verified

5.1 PIN Change

```
--- Change Security PIN ---
Enter Current PIN: 1234
Enter New 4-Digit PIN: 2222
Success: PIN updated successfully.

>>> Active: Badhon Kumar
1  Balance
2  Deposit
3  Withdraw
4  Transfer
5  History
6  Change PIN
7  Logout

Action:
```

Fig: Change PIN

6. Analytics & Profile Management

6.1 Analytics

The system provides bank-wide analytics including a summary of all accounts and top customer rankings.

```
GLOBAL BANK ANALYTICS

| Total Registered Users | 4 Persons |
| Total Bank Assets      | 7501.00 BDT |

--- TOP 5 VALUABLE CUSTOMERS ---
+--------+--------------+----------+-------------+
|   Rank | Name         |   Acc No | Balance     |
+========+==============+==========+=============+
|      1 | Badhon Kumar |     1001 | 4600.00 BDT |
+--------+--------------+----------+-------------+
|      2 | Biswas       |     1002 | 1901.00 BDT |
+--------+--------------+----------+-------------+
|      3 | B            |     1004 | 500.00 BDT  |
+--------+--------------+----------+-------------+
|      4 | Badhon       |     1005 | 500.00 BDT  |
+--------+--------------+----------+-------------+
```

Fig: Analytics — Bank Summary & Top Customers

6.2 Manage Profile

```
--- Manage Profiles ---

| Opt | Action |
|   1 | Update Profile |
|   2 | Close Account  |
|   3 | Back           |

Choice:
```

Fig: Manage Profile

6.3 Update Profile

```
--- Manage Profiles ---
╔═══╦═══════════╗
║   Opt ╒ Action
╟───╫───────────╢
    1 │ Update Profile
    2 │ Close Account
    3 │ Back
╚═══╩═══════════╝

Choice: 1
Enter Account Number to Update: 1002
New Name (Enter to skip): Badhon
New Phone (Enter to skip):
Profile updated!
```

Fig: Update Profile

6.4 Close Account

```
--- Manage Profiles ---
╔═══╦═══════════╗
║   Opt ╒ Action
╟───╫───────────╢
    1 │ Update Profile
    2 │ Close Account
    3 │ Back
╚═══╩═══════════╝

Choice: 2
Enter Account Number to Delete: 1002
Error: Account balance must be 0 BDT to close (Current: 1901.00 BDT)
```

Fig: Close Account

7. Data Persistence

- Data is stored in a CSV file (accounts_data.csv)
- All account records are loaded into memory when the program starts
- Data is saved back to CSV when the user exits the system

8. Challenges and Learnings

State Management	Managing data consistently across multiple modules required careful coordination using the DataStorage class as a central hub.
Input Validation	Ensuring valid, crash-free user input throughout all menu flows required thorough edge-case handling.
Modular Design	Dividing the project into clearly separated modules improved maintainability and facilitated effective teamwork.

9. Future Improvements

- Add a graphical user interface (GUI) using Tkinter or CustomTkinter
- Replace CSV storage with a relational database (SQLite or MySQL)
- Implement PIN encryption using secure hashing algorithms
- Introduce an auto-save system after every transaction

10. Conclusion

This project successfully demonstrates the application of core OOP principles, modular software architecture, and file-based data handling to build a real-world banking simulation system.The use of virtual environments and external libraries ensures the system is structured, scalable, and maintainable. The clean separation between entity classes, service managers, and the storage layer reflects industry best practices in software design.The project provides a strong foundation for future enhancements, including GUI-based interfaces and database integration, making it a practical and expandable solution for simulated banking operations.